Clustering and Initialization: k-means versus k-means++

Statistical Methods for Machine Learning — Experimental Project, A.Y. 2025/26

Aliza Aziz
Student ID: 82838A
Università degli Studi di Milano

November 6, 2026

Abstract

We study how the initialization of Lloyd's algorithm affects the quality and the variability of k-means clustering. We implement from scratch, in Python and numpy, the classical random initialization and the D^2 seeding of k-means++ (Arthur and Vassilvitskii, SODA 2007), and compare them over many paired random trials on a synthetic dataset that replicates the one used in the paper, on a 2-D toy dataset, and on a real high-dimensional multiclass dataset (UCI handwritten digits). On well-separated synthetic clusters, random seeding gets trapped in local minima whose cost is thousands of times the best one, while k-means++ finds the best solution in every run and converges in a single iteration. On the real dataset the advantage is small but consistent. Two extensions show that the gap grows with cluster separation and is essentially independent of dimension.

1 Introduction and problem setting

Given points $\mathcal{X} = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ and an integer k, k-means looks for k centres $C = \{c_1, \dots, c_k\}$ minimizing the *inertia* (also called potential, or within-cluster sum of squares)

$$\phi(C) = \sum_{i=1}^{n} \min_{c \in C} \|x_i - c\|^2. \tag{1}$$

In the language of the course this is empirical risk minimization with the squared loss over the class of "nearest-centre" quantizers. The problem is NP-hard and the objective is non-convex, so in practice it is attacked with *Lloyd's algorithm*: starting from some centres, repeat (i) assign every point to its closest centre, (ii) move every centre to the mean of its points. Both steps can only decrease ϕ, so the algorithm terminates, but only at a *local* minimum, and which one depends entirely on the starting centres. Two runs on the same data may therefore end in very different solutions.

Arthur and Vassilvitskii proposed k-means++, a randomized seeding rule (D^2 sampling) that picks well spread-out centres, and proved that the seeding *alone* already satisfies

$$\mathbb{E}[\phi(C_{\text{seed}})] \le 8(\ln k + 2)\, \phi_{\text{OPT}}, \tag{2}$$

while uniform random seeding has no such guarantee. Lloyd iterations can only improve on this. The goal of this project is to measure empirically how much this matters: (a) the distribution of final inertia over many runs, (b) its variability, and (c) the stability of the obtained partitions.

2 Algorithms and implementation

All code is in `src/kmeans.py`. Only numpy is used, to vectorize arithmetic and to sample random numbers; no clustering routine from any library is called. scikit-learn is used only to *load* the digits dataset and, in `tests/`, as an independent reference to check our Adjusted Rand Index.

Random initialization. k distinct data points are drawn uniformly without replacement (the usual "Forgy" scheme, also the baseline of the paper).

k-means++ (D^2 seeding). Let $D(x)$ be the distance of x to the closest centre already chosen.

Algorithm 1 D^2 seeding (k-means++)

$c_1 \leftarrow$ a data point chosen uniformly at random
for $i = 2, \dots, k$ **do**
    compute $D(x)^2$ for every $x \in \mathcal{X}$
    $c_i \leftarrow x$ with probability $D(x)^2 / \sum_{y \in \mathcal{X}} D(y)^2$
end for
return $\{c_1, \dots, c_k\}$

Points far from all current centres are much more likely to be chosen, so the centres spread over the data, but the rule is still random, which protects it from single outliers (a deterministic "farthest point" rule would always pick them). We keep a vector of the current $D(x)^2$ and update it with one distance computation per new centre, so seeding costs $O(nkd)$, the same as one Lloyd iteration.

Lloyd's algorithm. Shared by both methods, so any difference in the results is due to the seeding alone. Squared distances are computed in matrix form as $\|x\|^2 - 2x^\top c + \|c\|^2$. Iterations stop when the assignment no longer changes (an exact fixed point), with a safety cap of 300 iterations that was never reached. If a cluster becomes empty, its centre is moved to the point that is currently worst served; this keeps exactly k clusters and never increases ϕ. The same rule applies to both methods.

Correctness checks. `tests/test_kmeans.py` verifies that the fast distance formula matches a brute-force computation, that the objective never increases along Lloyd iterations, that D^2 seeding returns distinct points, that three obviously separated clusters are recovered exactly, and that our Adjusted Rand Index equals scikit-learn's on random labelings.

3 Datasets

The assignment requires a synthetic dataset with good clustering potential and a real-world high-dimensional multiclass dataset; we use both, plus a 2-D toy set for visualization and two synthetic families for the extensions.

- **Norm-25 (synthetic).** The synthetic set of the paper: 25 "true" centres drawn uniformly in $[0, 500]^{15}$, and 200 points per centre drawn from an isotropic Gaussian with unit variance ($n = 5000$, $d = 15$). The clusters are tiny compared with their distances, so the true partition is (with overwhelming probability) the optimal one for $k = 25$ and is known. This is the setting where initialization should matter the most, and the one where the paper reports the largest gains.
- **Blobs-2D (synthetic).** Nine Gaussian clusters ($\sigma = 0.6$) on a 3×3 grid with spacing 5, 150 points each. Used to *see* local minima.

- **UCI Digits (real).** The UCI "Optical Recognition of Handwritten Digits" data: 1797 images of 8×8 pixels, $d = 64$, 10 classes. It is an MNIST-like dataset (as suggested in the assignment), but small enough to run hundreds of clusterings in seconds, and it ships with scikit-learn so the project runs offline. All features are pixel intensities on the same scale 0–16, so we do not standardize them: rescaling would change the geometry k-means relies on.
- **Extensions.** (i) *Overlap:* 10 centres in $[0, 10]^{10}$ with cluster spread $\sigma \in \{0.25, 0.5, 1, 2, 4\}$, from clearly separated to strongly overlapping. (ii) *Dimension:* the Norm-25 generator with 100 points per cluster and $d \in \{2, 5, 15, 50, 100\}$.

Evaluation protocol and data leakage. Clustering is unsupervised and the object of study is the *optimization* of ϕ on a given dataset, so, as in the paper, each algorithm is run and evaluated on the same set of points; there is no prediction on unseen data to evaluate, hence no train/test split for the main experiments. The true labels are never given to the algorithms; they are used only afterwards to compute the agreement with the ground truth. As a sanity check that the centres are not "overfitting" the sample, we also learnt $k = 10$ centres on a random 80% of Digits and measured the per-point cost on the remaining 20% (Section 5.3).

4 Experimental design and metrics

For each dataset and each k we run **50 trials** of each method (20 trials per point in the extensions). Trial t uses the random seed t for both methods (a paired design), so every number in this report is reproducible by running `python run_experiments.py`. Following the paper we use $k \in \{10, 25, 50\}$ on Norm-25 and on Digits, which includes the "correct" value (25 and 10, respectively) as well as under- and over-clustering. We record:

- **seed inertia**: ϕ right after seeding, before any Lloyd step;
- **final inertia** and the **ratio** final/best, where "best" is the lowest inertia found by *any* run of either method on that configuration (a proxy for ϕ_{OPT});
- **variability**: standard deviation, coefficient of variation (CV = std/mean), worst ratio, and **hit%**, the fraction of runs within 0.1% of the best;
- **stability of the partition**: the mean Adjusted Rand Index (ARI) between the partitions produced by every pair of runs (1225 pairs). ARI equals 1 for identical partitions (up to renaming clusters) and is about 0 for random agreement. This measures stability of the *output*, not just of the cost;
- **ARI with the ground truth** when k equals the number of classes;
- number of Lloyd iterations and wall-clock time.

5 Results

5.1 Synthetic data: Norm-25

Table 1 and Figure 1 show the most striking result. For $k = 25$, **k-means++ reached the best solution in 50 out of 50 runs**, with zero variance, perfect agreement with the true clusters (ARI = 1) and with each other (pair ARI = 1). **Random seeding never reached it**: its final inertia is on average about **3200 times** the best (worst run ≈ 5100 times), with CV 26%, and its partitions agree with the truth only at ARI ≈ 0.70.

The reason is geometric. A run of random seeding succeeds only if every one of the 25 equally sized clusters receives exactly one initial centre, which happens with probability $25!/25^{25} \approx 1.7 \times$

Table 1: Results on `Norm-25` over 50 paired trials per method. Ratio = final inertia divided by the best inertia found by any run; *hit* = runs within 0.1% of the best; pair ARI = mean agreement between partitions of two runs.

k	method	seed inertia	final mean	final std	CV%	ratio mean	ratio max	hit%	iters	pair ARI	ARI truth
10	random	1.327e9	7.398e8	5.913e7	7.99	1.224	1.436	0	6.7	0.355	–
10	k-means++	1.122e9	6.790e8	3.688e7	5.43	1.123	1.263	2	1.5	0.421	–
25	random	5.871e8	2.375e8	6.096e7	25.66	3,181.805	5,079.173	0	15.6	0.561	0.697
25	k-means++	1.492e5	7.466e4	0	0.00	1.000	1.000	100	1.0	1.000	1.000
50	random	2.038e8	3.912e7	3.768e7	96.33	557.846	2,036.614	0	20.0	0.586	–
50	k-means++	1.273e5	7.033e4	1.224e2	0.17	1.003	1.007	12	18.8	0.640	–

Table 2: Results on `Digits` over 50 paired trials per method. Ratio = final inertia divided by the best inertia found by any run; *hit* = runs within 0.1% of the best; pair ARI = mean agreement between partitions of two runs.

k	method	seed inertia	final mean	final std	CV%	ratio mean	ratio max	hit%	iters	pair ARI	ARI truth
10	random	2.280e6	1.188e6	2.673e4	2.25	1.020	1.090	24	20.1	0.753	0.618
10	k-means++	2.250e6	1.188e6	2.580e4	2.17	1.019	1.088	12	19.0	0.751	0.610
25	random	1.684e6	9.034e5	9.664e3	1.07	1.023	1.059	0	22.6	0.643	–
25	k-means++	1.626e6	9.009e5	1.002e4	1.11	1.020	1.057	2	20.3	0.655	–
50	random	1.319e6	7.363e5	9.383e3	1.27	1.030	1.062	2	18.8	0.545	–
50	k-means++	1.288e6	7.319e5	7.093e3	0.97	1.024	1.043	0	18.4	0.562	–

10^{-10}. Otherwise some true cluster gets two centres and another gets none, and Lloyd's algorithm cannot repair this: a centre that sits inside a cluster 500 units away from the empty one never "sees" its points, so two far-apart true clusters end up merged and the cost explodes (each merged pair contributes a squared distance of order $500^2 \cdot d/6$ per point). With D^2 seeding, once a cluster contains a centre its points have $D(x)^2 \approx d\sigma^2 = 15$, whereas points of uncovered clusters have $D(x)^2$ in the order of 10^5–10^6; the next centre is thus almost certainly drawn from an uncovered cluster. This is why k-means++ is *already* essentially optimal after seeding: its seed inertia is 2.0 times the best (each point is measured from a random point of its cluster instead of the mean, which doubles the expected cost of a Gaussian cluster), well inside the bound (2), which for $k = 25$ is $8(\ln 25 + 2) \approx 41.8$. One Lloyd iteration then moves every centre to its cluster mean and the algorithm stops: **1.0 iteration on average against 15.6** for random seeding (Figure 2, left). The seed inertia of random seeding, by contrast, is about 7900 times the best.

For $k = 50$ (more centres than true clusters) random seeding still fails most of the time (CV 96%, mean ratio ≈ 560), because the chance of leaving some cluster empty remains high; a few lucky runs reach a ratio close to 1, which is why its box spans three orders of magnitude. k-means++ is within 0.7% of the best in every run. Here the optimum is not unique-looking anymore (the 25 extra centres can split clusters in many nearly equivalent ways), which explains why the pair ARI of k-means++ is only 0.64 even though the cost is almost constant: *different partitions with the same cost* are a form of instability that the inertia alone does not reveal. For $k = 10$ (fewer centres than clusters) both methods must merge clusters; k-means++ is still better (mean ratio 1.12 vs 1.22) but neither finds the best grouping reliably.

5.2 Visualizing local minima: Blobs-2D

Figure 3 shows the worst of 50 runs of each method on the 2-D grid. In both cases the failure has the same shape: two centres share one blob and a single centre is left halfway between two blobs. Random seeding reached the best solution in 30% of the runs, k-means++ in **76%**; mean ratios are 2.65 and 1.50, and the mean ARI with the truth is 0.88 vs 0.96. This is an honest negative aspect: k-means++ is not a guarantee of optimality, only a guarantee in expectation, and with clusters this close (spacing 5, $\sigma = 0.6$) the D^2 distribution still puts some mass on already covered blobs. Its worst run is about as bad as the worst random run; what changes is how often it happens.

Table 3: Results on `Blobs-2D` over 50 paired trials per method. Ratio = final inertia divided by the best inertia found by any run; *hit* = runs within 0.1% of the best; pair ARI = mean agreement between partitions of two runs.

k	method	seed inertia	final mean	final std	CV%	ratio mean	ratio max	hit%	iters	pair ARI	ARI truth
9	random	1.161e4	2.553e3	1.232e3	48.26	2.651	4.930	30	9.3	0.788	0.878
9	k-means++	3.509e3	1.444e3	8.912e2	61.71	1.500	4.815	76	4.7	0.926	0.962

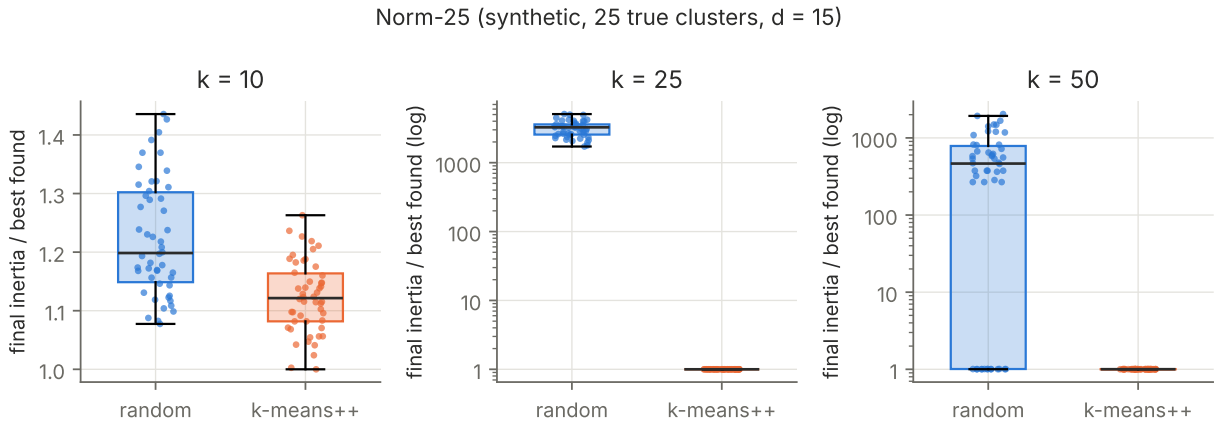


Figure 1: Norm-25: distribution of final inertia relative to the best found, 50 runs per method (each dot is one run). Note the log scale for $k = 25, 50$.

5.3 Real-world data: UCI Digits

On Digits (Table 2, Figure 4) the picture is very different: **all runs of both methods end within 9% of the best**, and the mean ratios are close (1.020 vs 1.019 for $k = 10$; 1.023 vs 1.020 for $k = 25$; 1.030 vs 1.024 for $k = 50$). k-means++ has a slightly lower mean and, for larger k, a lower worst case (ratio max 1.043 vs 1.062 at $k = 50$) and lower CV (0.97% vs 1.27%). Its seeding cost is lower in all cases, but only by a few percent (seed ratio 1.93 vs 1.96 for $k = 10$). The pair ARI is practically identical (≈ 0.75 for $k = 10$), so the partitions found are as unstable with either seeding, and the ARI with the true digit classes is ≈ 0.61 for both: k-means captures much of the digit structure but cannot separate classes such as 1, 7 and 9 that are not compact, well-separated blobs in pixel space. Convergence speed is also similar (about 19–20 iterations, Figure 2, right), and the seeding overhead of k-means++ is negligible at this scale (all runs take tens of milliseconds).

Why so little difference? Real data in 64 dimensions does not consist of well separated blobs: the classes overlap and the inertia landscape has many local minima of very similar cost. Random points already fall in every dense region, so there is little room for a clever seeding to help. This is consistent with the paper, where the gains on real datasets are smaller than on Norm-25. The held-out check confirms that the centres generalize: with $k = 10$ the average per-point cost is 658.3 (train) vs 688.1 (test) for random seeding and 656.3 vs 684.1 for k-means++; the small train/test gap is the same for both methods, so better seeding does not mean overfitting.

5.4 Stability across seeds

Figure 5 summarizes stability from the point of view of the *output*. When the data has a clear cluster structure that matches k (Norm-25, Blobs-2D), k-means++ is far more stable: on Norm-25 it returns the very same partition every time, while two random runs agree only at ARI ≈ 0.56. On Digits both seedings are equally unstable, because the instability comes from the data (many near-optimal partitions), not from the seeding.

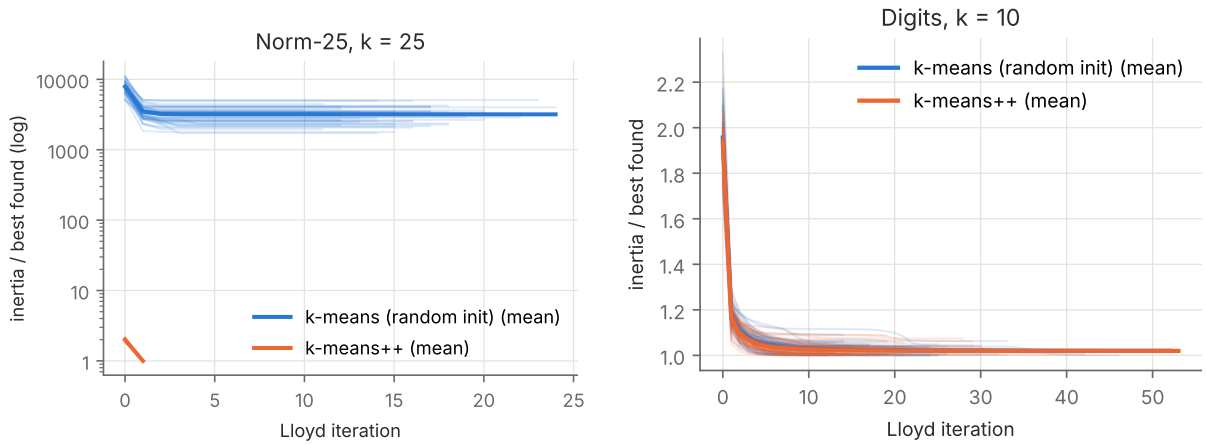


Figure 2: Inertia (relative to the best) along Lloyd iterations; thin lines are single runs, thick lines are means. Iteration 0 is the cost of the seeding.

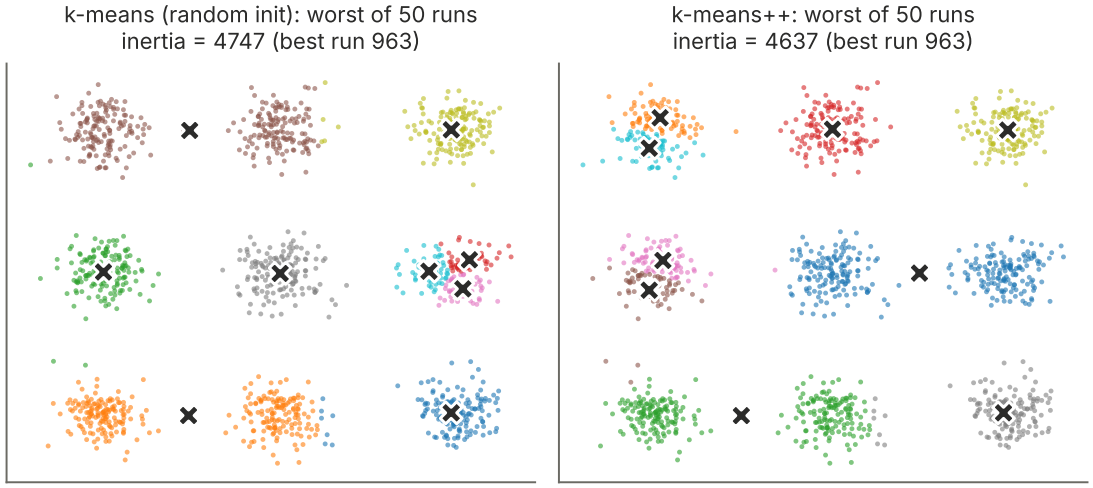


Figure 3: Blobs-2D, $k = 9$: worst run out of 50 for each method. Crosses are the final centres, colors the final clusters.

5.5 Extensions: overlapping clusters and dimensionality

Overlap. With well-separated clusters ($\sigma = 0.25$) random seeding ends on average at 10.6 times the best cost against 1.6 for k-means++ (ARI with truth 0.78 vs 0.96). As σ grows the advantage shrinks, and from $\sigma = 1$ on the two methods are indistinguishable; at $\sigma = 4$ the clusters are mixed so much that both recover the truth poorly (ARI ≈ 0.34) while the cost is within 0.6% of the best for both. In other words, the value of careful seeding is largest exactly when there *is* a clear structure to find, which is the same conclusion as Norm-25 vs Digits. Note that k-means++ is not perfect even at $\sigma = 0.25$ (ratio 1.6), because with only 10 centres in a small cube some true centres are close to each other.

Dimension. With the Norm-25 generator, k-means++ is optimal in every run for every $d \geq 5$, while random seeding gets worse as d grows (mean ratio from about 280 at $d = 2$ to about 4500 at $d = 100$). The ARI of random seeding stays around 0.7 for all d: the *probability* of a bad seeding does not depend on d (it is the same "balls into bins" event), but the *price* of a merged pair of clusters grows linearly with d, since the squared distance between two random points of the cube does. At $d = 2$ some true centres happen to be close, so k-means++ loses a little (ratio 4.0, ARI 0.99).

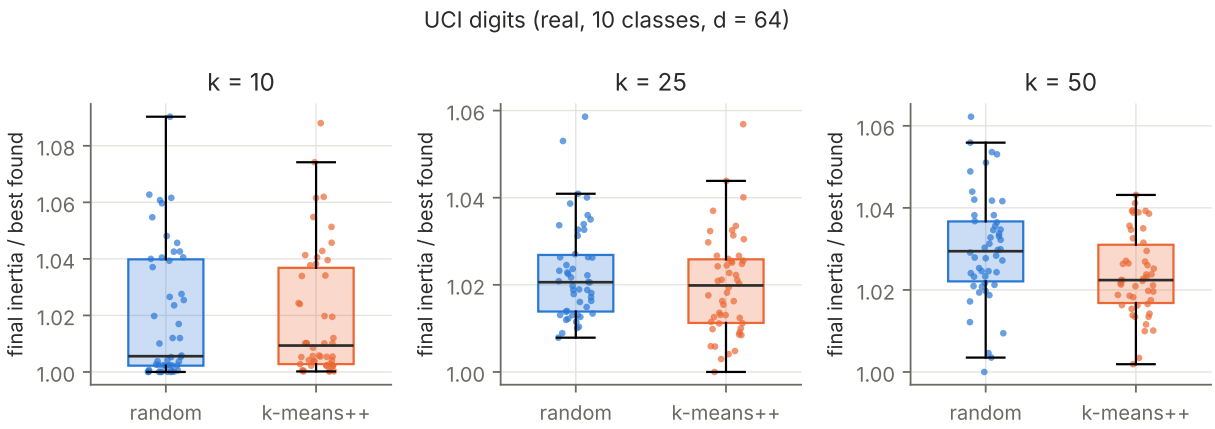


Figure 4: UCI Digits: distribution of final inertia relative to the best found (linear scale; note how narrow the range is compared with Figure 1).

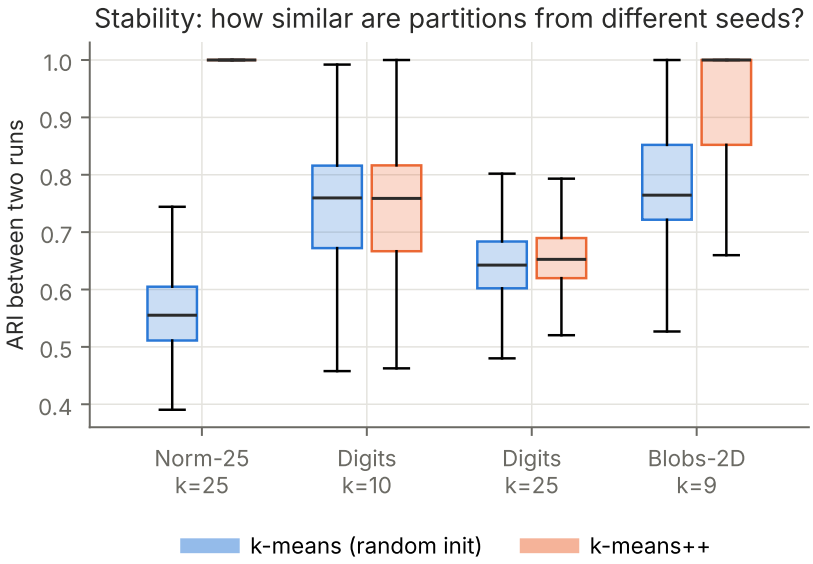


Figure 5: ARI between the partitions of every pair of runs (1225 pairs per box).

6 Discussion and conclusions

What we observed.

1. Initialization can change the *final* k-means cost by more than three orders of magnitude on data with well separated clusters, and Lloyd's algorithm cannot fix a bad start: it only moves centres locally.

2. k-means++ removes this failure mode almost completely: on Norm-25 it found the best solution in 100% of runs, in a single Lloyd iteration, and its seeding cost is far below the theoretical bound $8(\ln k + 2)$.

3. Its benefits are much smaller on real, overlapping data (Digits), where all local minima have similar cost; there it is still never worse on average and has a slightly better worst case.

4. The cost alone can hide instability: for $k = 50$ on Norm-25 and on Digits, runs with almost the same inertia return clearly different partitions.

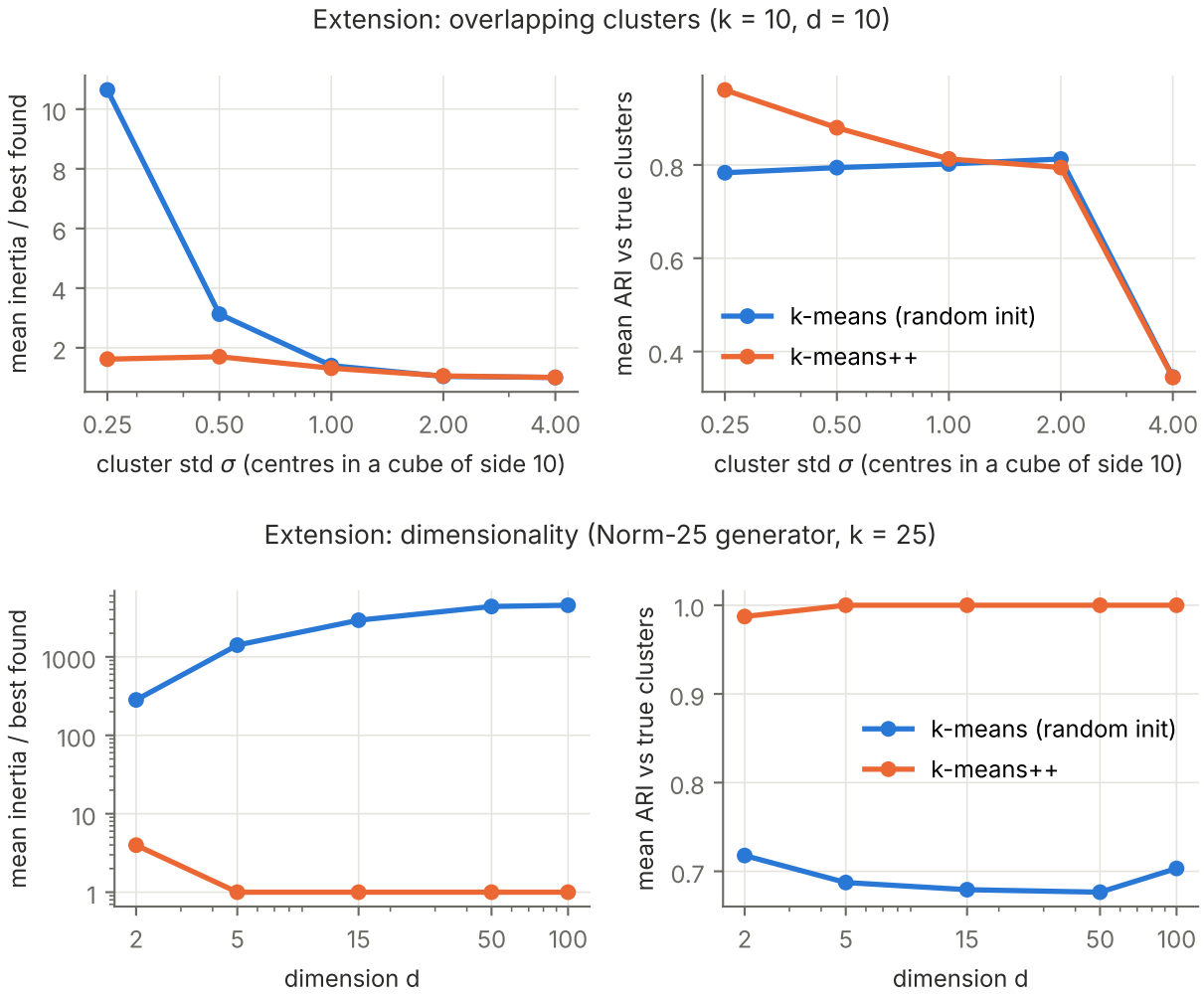


Figure 6: Top: increasing cluster overlap. Bottom: increasing dimension. Means over 20 paired runs per point.

Limitations. The "best" inertia is the best found, not a certified optimum; 50 trials estimate the tails of the distribution only roughly; Digits is one real dataset of moderate size, and we did not tune the number of clusters or compare with other remedies (multiple restarts, which scikit-learn uses by default, or greedy k-means++ that samples several candidates per step). Timing comparisons at this scale are dominated by Python overhead and should not be read as general conclusions. Finally, k-means++ does not fix the deeper limitations of the k-means objective itself (spherical, equally sized clusters), which is visible in the modest ARI on Digits.

Reproducibility. `pip install -r requirements.txt`, then `python run_experiments.py` regenerates every figure, every table of this report (`report/tables.tex`) and the raw results in `results/` in under a minute; all randomness is controlled by explicit seeds.

References

- D. Arthur and S. Vassilvitskii. *k-means++: The Advantages of Careful Seeding.* SODA 2007.
- S. Lloyd. *Least squares quantization in PCM.* IEEE Trans. Information Theory, 1982.
- L. Hubert and P. Arabie. *Comparing partitions.* Journal of Classification, 1985.

- E. Alpaydin and C. Kaynak. *Optical Recognition of Handwritten Digits*. UCI Machine Learning Repository, 1998.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.